

Tarea 1 Manejo de Datos

David Angel Gonzalez Hidalgo

Diego Zaid Reyes Miranda

Luis Daniel Arévalo Esquivel

Parte 1: Investigación

Reglas PEP8:

1. Usa 4 espacios por nivel. Nunca mezcles espacios y tabuladores.

Antes:

```
def saludar(nombre):
```

```
    if nombre:
```

```
        print("Hola", nombre)
```

Después:

```
def saludar(nombre):
```

```
    if nombre:
```

```
        print("Hola", nombre)
```

2. Longitud de línea: Máximo 79 caracteres en código y 72 en comentarios/docstrings.

Antes:

```
resultado = calcular_precio_total(producto, cantidad, descuento, impuesto, envio)
```

Después:

```
resultado = calcular_precio_total(
```

```
    producto,
```

```
    cantidad,
```

```
    descuento,
```

```
    impuesto,
```

envio,

)

3. Espacios en expresiones: Coloca espacios alrededor de operadores (`a = b + c`), pero evita espacios dentro de paréntesis o corchetes.

Antes:

```
resultado=a+b*c
```

```
lista = [ 1, 2, 3 ]
```

Después:

```
resultado = a + b * c
```

```
lista = [1, 2, 3]
```

4. Imports: Hazlos al inicio del archivo, separados en grupos. Nunca uses `import *`.

Antes:

```
def calcular():
```

```
    import math
```

```
    from random import *
```

```
    return sqrt(25)
```

Después:

```
import math
```

```
import random
```

```
def calcular():
```

```
    return math.sqrt(25)
```

5. Nombres:

- Variables y funciones: `snake_case` (ej. `calculate_total`).

- Clases: `CamelCase` (ej. `CustomerOrder`).
- Constantes: mayúsculas con guiones bajos (`MAX_SIZE`).

Antes:

```
NombreUsuario = "Diego"
```

```
def CalcularTotal(precio, cantidad):
```

```
    return precio * cantidad
```

Después:

```
nombre_usuario = "Diego"
```

```
def calcular_total(precio, cantidad):
```

```
    return precio * cantidad
```

6. Comentarios: Claros y útiles. Los comentarios en línea van después de `#`, los de bloque encima del código.

Antes:

```
#calcula el total
```

```
total=precio*cantidad #multiplica precio por cantidad
```

Después:

```
# Calcula el total de la compra.
```

```
total = precio * cantidad
```

```
# Aplica el descuento correspondiente.
```

```
total = total * (1 - descuento)
```

7. Docstrings: Usa triple comillas dobles (`"""`) para describir funciones, clases y módulos siguiendo PEP 257.

Antes:

```
def calcular_area(base, altura):
```

```
# Calcula el área
```

```
return base * altura / 2
```

Después:

```
def calcular_area(base, altura):
```

```
    """Calcula el área de un triángulo.
```

Args:

base: Base del triángulo.

altura: Altura del triángulo.

Returns:

El área del triángulo.

```
    """
```

```
return base * altura / 2
```

8. Comas finales: Úsalas en listas largas para facilitar futuras modificaciones.

Antes:

```
usuarios = [
```

```
    "Ana",
```

```
    "Carlos",
```

```
    "Diego"
```

```
]
```

Después:

```
usuarios = [
```

```
    "Ana",
```

```
    "Carlos",
```

```
    "Diego",
```

]

9. Argumentos de métodos: Siempre usa `self` en métodos de instancia y `cls` en métodos de clase.

Antes:

```
class Persona:

    def saludar(nombre):

        print("Hola", nombre)
```

Después:

```
class Persona:

    def saludar(self, nombre):

        print("Hola", nombre)
```

10. Codificación de archivos: Usa UTF-8 por defecto para asegurar compatibilidad internacional.

Antes:

```
class Persona:

    @classmethod

    def crear(nombre):

        return Persona(nombre)
```

Después:

```
class Persona:

    @classmethod

    def crear(cls, nombre):

        return cls(nombre)
```

¿Qué es SOLID?:

Las reglas "SOLID" son un conjunto de principios orientados a la creación de programas de manera limpia, ordenada y flexible postulados por Robert C. Martin.

El nombre SOLID deriva del acrónimo formado por el nombre de las 5 reglas que componen este grupo, los cuales son:

- Single Responsibility Principle (Principio de responsabilidad única): Este principio busca evitar el uso de clases que hacen muchas cosas y tienen diferentes razones para cambiar. Esto a través de la asignación de una tarea concreta por clase.

Ejemplo:

Código malo: la clase mezcla cálculo y persistencia

```
class Asegurado:
    def __init__(self, nombre, sa, k):
        self.nombre = nombre
        self.sa = sa
        self.k = k

    def calcular_prima(self):
        return self.sa * self.k / 1000

    def guardar_en_archivo(self, ruta):
        with open(ruta, "a") as f:
            f.write(f"{self.nombre}: {self.calcular_prima()}\n")
    ...
```

Código bueno: se separa el cálculo de la persistencia:

```
class Asegurado:
    def __init__(self, nombre, sa, k):
        self.nombre = nombre
        self.sa = sa
        self.k = k

    def calcular_prima(self):
        return self.sa * self.k / 1000

class ExportadorCarnet:
    def guardar(self, asegurado, ruta):
        with open(ruta, "a") as f:
            f.write(f"{asegurado.nombre}: {asegurado.calcular_prima()}\n")
    ...
```

- Open/Closed Principle (Principio de abierto/cerrado): Aquí se propone que las clases usadas deben poder alterar su comportamiento sin afectar el funcionamiento de un

código ya funcional. Esto para evitar modificar un programa ya existente cada vez que se le agregue algo a una clase.

Ejemplo:

Código malo: agregar un nuevo tipo de asegurado obliga a modificar la función

```
def calcular_factor(sexo, edad):
    if sexo == "F":
        return 1.5
    elif sexo == "M":
        return 2.0
    # cada nuevo caso requiere editar esta función
    ...
```

Código bueno: se extiende agregando subclases, sin tocar el código existente
from abc import ABC, abstractmethod

```
class FactorEdadStrategy(ABC):
    @abstractmethod
    def obtener_factor(self, edad):
        ...

class FactorFemenino(FactorEdadStrategy):
    def obtener_factor(self, edad):
        return 1.5 if edad <= 25 else 1.7

class FactorMasculino(FactorEdadStrategy):
    def obtener_factor(self, edad):
        return 2.0 if edad <= 25 else 2.3
    ...
```

- Liskov Substitution Principle (Principio de sustitución de Liskov): Para casos donde una subclase no pueden comportarse concretamente como sus clases padre. Este punto propone que una clase hija puede sustituir algunos objetos de la clase padre sin alterar el comportamiento del programa

Ejemplo:

Código malo: la subclase no respeta el contrato de la clase base

```
class FactorEdadStrategy:
    def obtener_factor(self, edad):
        return 1.0

class FactorInvalido(FactorEdadStrategy):
    def obtener_factor(self, edad):
```

```
        raise NotImplementedError("No implementado") # rompe el contrato
    ...
```

Código bueno: toda subclase cumple el mismo contrato y puede sustituirse

```
class FactorEdadStrategy(ABC):
    @abstractmethod
    def obtener_factor(self, edad):
        """Siempre regresa un factor numérico válido."""
class FactorFemenino(FactorEdadStrategy):
    def obtener_factor(self, edad):
        return 1.5 if edad <= 25 else 1.7

class FactorMasculino(FactorEdadStrategy):
    def obtener_factor(self, edad):
        return 2.0 if edad <= 25 else 2.3
# Cualquiera de las dos puede usarse donde se espera un FactorEdadStrategy
...
```

- Interface Segregation Principle (Principio de segregación de interfaces): Este principio nos habla del uso de interfaces pequeñas y específicas en lugar de interfaces grandes que obligan a las clases del programa a implementar métodos que no necesitan.

Ejemplo:

Código malo: una sola interfaz obliga a implementar métodos innecesarios

```
class ServicioAsegurado(ABC):
    @abstractmethod
    def calcular_prima(self): ...
    @abstractmethod
    def exportar_pdf(self): ...
    @abstractmethod
    def convertir_moneda(self): ...

class AseguradoBasico(ServicioAsegurado):
    def calcular_prima(self):
        return 100
    def exportar_pdf(self):
        raise NotImplementedError # no lo necesita
    def convertir_moneda(self):
        raise NotImplementedError # no lo necesita
    ...
```

Código bueno: interfaces pequeñas y específicas


```

class CalculaPrima(ABC):
    @abstractmethod
    def calcular_prima(self): ...

class ConvierteMoneda(ABC):
    @abstractmethod
    def convertir_moneda(self, monto): ...

class AseguradoBasico(CalculaPrima):
    def calcular_prima(self):
        return 100
...

```

- Dependency inversion Principle (Principio de inversión de dependencias): Para evitar ligar el código a una clase concreta que dificulte una variación en los métodos de las subclases y demás partes del código, este principio propone la dependencia a abstracciones para las características necesarias en cada caso.

Ejemplo:

Código malo: la clase depende directamente de una implementación concreta

```

class CalculadoraPrima:
    def convertir_a_usd(self, monto_mxn):
        tasa = 21.13 # acoplado directamente al valor concreto
        return monto_mxn / tasa
...

```

Código bueno: se depende de una abstracción inyectada

```

class ServicioTasaCambio(ABC):
    @abstractmethod
    def obtener_tasa(self): ...

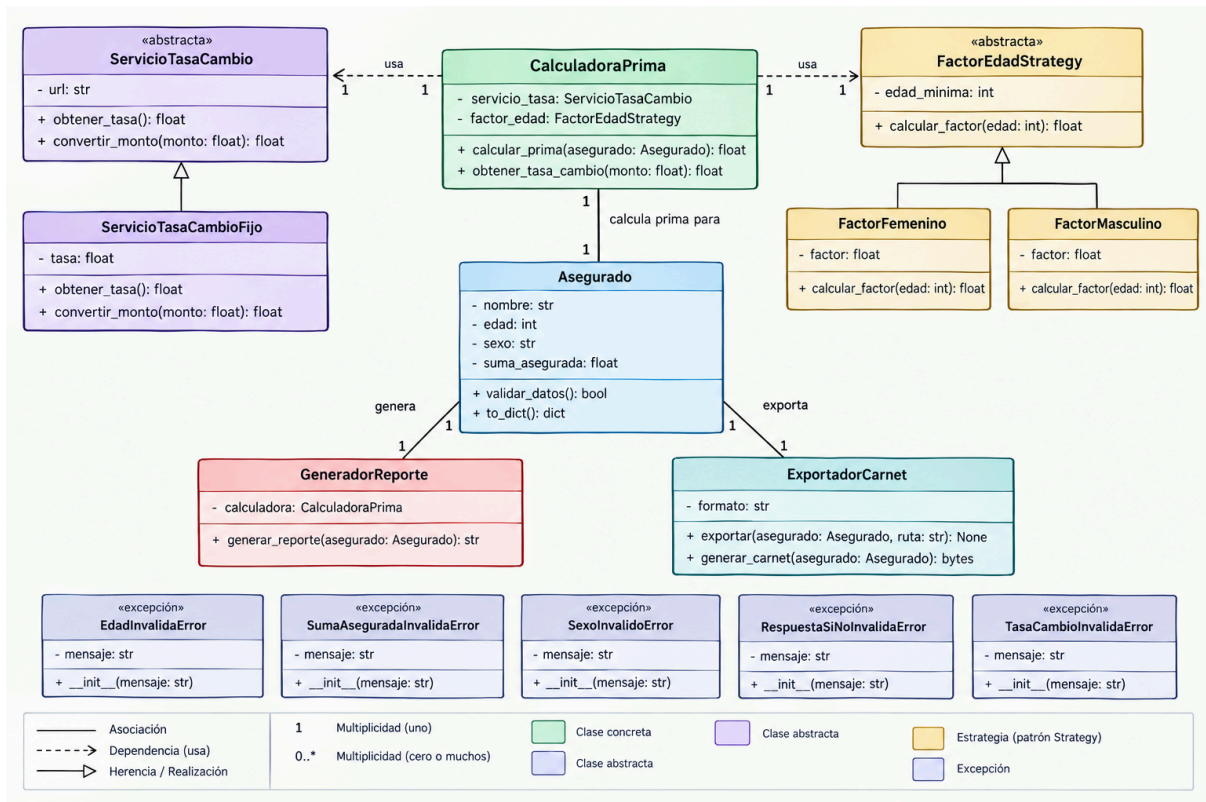
class ServicioTasaCambioFijo(ServicioTasaCambio):
    def __init__(self, tasa=21.13):
        self.tasa = tasa
    def obtener_tasa(self):
        return self.tasa

class CalculadoraPrima:
    def __init__(self, servicio_tasa: ServicioTasaCambio):
        self.servicio_tasa = servicio_tasa

    def convertir_a_usd(self, monto_mxn):
        return monto_mxn / self.servicio_tasa.obtener_tasa()
...

```

Parte 2: Diagrama de clases (problema de aseguradora).



Fuentes:

Python Software Foundation. (2001). PEP 8 – Style Guide for Python Code. Recuperado de <https://peps.python.org/pep-0008/>

Ryware. (2026, julio 18). Principios SOLID: guía práctica para el diseño de sistemas. Recuperado de <https://ryware.com/blog/principios-solid-guia-practica>